

Reviewer Pack — Minimal Rank of Quotient Algebras over Tseitin Circuit Valua...

Entry 0be8c48e1f19 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 14:22:01 UTC

Minimal Rank of Quotient Algebras over Tseitin Circuit Valuations

Entry ID: 0be8c48e1f19 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 14:22:01 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory (Quotient Algebras) **Field B** (complexity object): Complexity Theory: Tseitin Circuit Size

Statement:

For any given positive integer n , there exists a constant $C(n)$ such that the minimal rank of the quotient algebra A over the Tseitin circuit valuation for a randomly generated CNF formula with n variables is upper bounded by $C(n)$. Specifically, $\text{rank}(A) \leq C(n) * \log^2(n)$.

Rationale (proposer's reasoning):

Quotient algebras provide a representation-theoretic framework that can capture structural properties of boolean functions. The Tseitin circuit valuation corresponds to the size of minimal circuits for boolean functions. If the conjecture holds, it would imply that the complexity of Tseitin circuits is well-approximated by their quotient algebra rank, which could potentially lead to new insights into circuit lower bounds.

Taxonomy category: GCT_DET_PERM (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a36f84f4cebdbdbf

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all n from 1 to 40, the average minimal rank of quotient algebras over Tseitin circuit valuations from randomly generated CNF formulas with n variables is less than or equal to $C(n) * \log^2(n)$, where $C(n)$ is a constant and 'log' denotes the natural logarithm. The conjecture is falsified if any n in this range has an average minimal rank greater than $C(n) * \log^2(n)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.70	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'minimal rank of quotient algebras' AND 'Tseitin circuit size' - 'Representation Theory' AND 'quotient algebra' AND 'CNF formula' AND 'complexity theory' - Tseitin circuit valuation AND 'algebraic geometry' AND 'logarithmic bound'

Top relevant hits considered: - [<http://arxiv.org/abs/2308.14302v2>] Finite simple characteristic quotients of the free group of rank 2 - [<http://arxiv.org/abs/2412.20260v2>] Functorial, operadic and modular operadic combinatorics of circuit algebras - [<http://arxiv.org/abs/2108.04557v3>] Brauer diagrams, modular operads, and a graphical nerve theorem for circuit algebras - [<http://arxiv.org/abs/2011.03611v1>] Twisted and folded Auslander-Reiten quivers and applications to the representation theory of quantum affine algebras - [<http://arxiv.org/abs/2212.10781v4>] On J -folded alcove paths and combinatorial representations of affine Hecke algebras - [<http://arxiv.org/abs/2107.0331>] Coxeter combinatorics for sum formulas in the representation theory of algebraic groups - [<http://arxiv.org/abs/1211.0398v1>] Extending valuations to formal completions - [<http://arxiv.org/abs/2407.1795>] Pruning Boolean d-DNNF Circuits Through Tseitin-Awareness

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=243.8s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(2 ** n - 1):
            clause = [random.randint(1, n), random.randint(-n, -1)]
            clauses.append(clause)
        return clauses

    def tseitin_valuation(cnf):
        literals = set()
        for clause in cnf:
            literals.update(abs(lit) for lit in clause)

        valuation = {}
```

```

    for literal in literals:
        if literal > 0:
            valuation[literal] = random.randint(0, 1)
        else:
            valuation[-literal] = 1 - valuation[-literal]

    return valuation

def quotient_algebra_rank(valuation):
    rank = 0
    for literal in valuation:
        if valuation[literal] == 1:
            rank += 1
    return rank

n_values = [5, 10, 15, 20, 30, 40]
total_rank = 0
instances_tested = 0

for n in n_values:
    for _ in range(5): # Test each n with 5 different instances
        cnf = generate_cnf(n)
        valuation = tseitin_valuation(cnf)
        rank = quotient_algebra_rank(valuation)
        total_rank += rank
        instances_tested += 1

average_rank = total_rank / instances_tested
C_n = 1 # Placeholder constant, adjust as needed
expected_bound = C_n * (math.log(n) ** 2)

conjecture_holds = average_rank <= expected_bound
counterexample = "" if conjecture_holds else f"rank={average_rank}, expected={expected_bound}"

return {
    "metric_name": "quotient_algebra_rank",
    "metric_value": average_rank,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 30)) # Default to first 29 primes if

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):

```

```

print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction=1.0")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction:.2f}")
else:
    first_failing_seed = next((seed for seed, result in enumerate(results) if not result["conjecture"]), None)
    print(f"RESULT: FALSIFIED counterexample=\"{results[first_failing_seed]['counterexample']}\"")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means we cannot verify the conjecture for all n from 1 to 40. | next: Run the test again with increased time limits or optimize the code to ensure it completes within a reasonable timeframe.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16765
2	preregistration	ollama_remote	glm4:latest	0	0	6115
3	novelty	ollama_remote	glm4:latest	0	0	5095
4	novelty	ollama_remote	glm4:latest	0	0	6182
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11817
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9770
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10468
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8712
9	judge	ollama_remote	glm4:latest	0	0	55114

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 130038 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/0be8c48e1f19.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0be8c48e1f19.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/0be8c48e1f19.tar.gz` (if generated)